LAPORAN PRAKTIKUM

STRUKTUR DATA

Queue

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 30 APRIL 2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul "**Queue**". Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Queue.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 30 April 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM 1
KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Kode Program 2
2.2.1 IterasiQueue 2
2.2.2 QueueArray 2
2.2.3 QueueArrayDriver 4
2.2.4 QueueLinkedList 5
2.2.5 ReverseData 5
BAB III KESIMPULAN DAN SARAN 7
3.1 Kesimpulan 7
DAFTAR PUSTAKA 8

DAFTAR GAMBAR

gambar 1 2
gambar 2 4
gambar 3 5
gambar 4 6

BAB I
PENDAHULUAN

1.1 Latar Belakang

- Struktur data adalah cara untuk menyimpan dan mengatur data sehingga Anda dapat menggunakannya secara efisien. Dalam Java, Struktur Data Queue adalah konsep fundamental dalam ilmu komputer yang digunakan untuk menyimpan dan mengelola data dalam urutan tertentu. Sistem ini mengikuti prinsip First In First Out (FIFO), di mana elemen pertama yang ditambahkan ke antrian adalah elemen pertama yang dikeluarkan.

  Prinsip FIFO menyatakan bahwa elemen pertama yang ditambahkan ke antrian akan menjadi elemen pertama yang dikeluarkan atau diproses. Jadi, antrian seperti barisan orang yang menunggu untuk membeli tiket, di mana orang pertama dalam barisan adalah orang pertama yang dilayani.

1.2 Tujuan Praktikum

- Memahami konsep *Abstract Data Type* (ADT) *Queue* dalam pemrograman.
- Mampu merancang dan mengimplementasikan *Queue* menggunakan bahasa Java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan *Queue* dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Kelas Queue merepresentasikan antrian objek *first-in-first-out* (FIFO) yang mana penyisipan dilakukan di ujung belakang dan penghapusan dilakukan dari ujung depan. Ada beberapa operasi dasar queue yaitu enqueue untuk memasukkan elemen kedalam antrian, dequeue menghapus elemen antrian, getFront mengambil data terdepan tanpa menghapus, getRear mengambil data belakang tanpa menghapus, dan masih banyak lagi

2.2 Kode Program

2.2.1 IterasiQueue

```
package pekan4_2511533007;

import java.util.*;

public class IterasiQueue_2511533007 {
        public static void main(String args[]) {

                Queue<String> q_3007 = new LinkedList<>();

                q_3007.add("Praktikum");
                q_3007.add("Struktur");
                q_3007.add("Data");
                q_3007.add("Dan");
                q_3007.add("Algoritma");
                Iterator<String> iterator_3007 = q_3007.iterator();
                while (iterator_3007.hasNext()) {
                        System.out.print(iterator_3007.next() + " ");
                }
        }
}
```

Kode Program 2.1

Kode program merupakan implementasi struktur data Queue menggunakan kelas LinkedList untuk menyimpan sekumpulan data string. Program menginisialisasi objek antrean bernama q_3007 dan mengisi parameter berupa kata menggunakan metode add(). Untuk menampilkan isi Queue, program memanfaatkan interface Iterator yang memungkinkan untuk mengulang melalui koleksi. Melalui perulangan while dan pengecekan kondisi hasNext(), setiap elemen diambil menggunakan metode next() dan diprint.

```
Praktikum Struktur Data Dan Algoritma
```

gambar 1

2.2.2 QueueArray

```
package pekan4_2511533007;

public class QueueArray_2511533007 {
        int front_3007, rear_3007, size_3007;
        int capacity_3007;
        int array_3007[];
```

```java
        public QueueArray_2511533007(int capacity_3007) {
                this.capacity_3007 = capacity_3007;
                front_3007 = this.size_3007 = 0;
                rear_3007 = capacity_3007 - 1;
                array_3007 = new int [this.capacity_3007];
        }

        boolean isFull_3007(QueueArray_2511533007 queue_3007) {
                return (queue_3007.size_3007 == queue_3007.capacity_3007);
        }

        boolean isEmpty_3007(QueueArray_2511533007 queue_3007) {
                return (queue_3007.size_3007 == 0);
        }

        void enqueue_3007(int item_3007 ) {
                if (isFull_3007(this))
                        return;
                this.rear_3007 = (this.rear_3007 + 1) % this.capacity_3007;
                this.array_3007[this.rear_3007] = item_3007;
                this.size_3007 = this.size_3007 + 1;
                System.out.println(item_3007 + " enqueued to queue");
        }
        int dequeue_3007() {
                if (isEmpty_3007(this))
                        return Integer.MIN_VALUE;
                int item_3007 = this.array_3007[this.front_3007];
                this.front_3007 = (this.front_3007 + 1) % this.capacity_3007;
                this.size_3007 = this.size_3007 - 1;
                return item_3007;
        }
        int front_3007() {
                if (isEmpty_3007(this))
                        return Integer.MIN_VALUE;

                return this.array_3007[this.front_3007];
        }
        int rear_3007() {
                if (isEmpty_3007(this))
                        return Integer.MIN_VALUE;
                return this.array_3007[this.rear_3007];
        }
        //mencetak elemen antrian
        void display_3007() {
                int i;
                if (front_3007 == rear_3007) {
                        System.out.printf("\nAntrian Kosong\n");
                        return;
                }
                //kunjungi dari belakang dan cetak
                for (i = front_3007; i < rear_3007; i++) {
                        System.out.printf(" %d <-- ", array_3007[i]);
                }
                return;
        }

}
```

Kode Program 2.2

Kode tersebut mengimplementasikan struktur data queue melingkar (circular queue). Circular Queue adalah versi lanjutan dari Queue linier di mana posisi terakhir dihubungkan kembali ke posisi pertama, membentuk lingkaran. Hal ini memungkinkan Queue untuk memanfaatkan memori secara efisien dengan menggunakan kembali ruang yang dibebaskan setelah elemen dikeluarkan. Menggunakan array dalam Java, dengan atribut utama front_3007, rear_3007, size_3007, dan capacity_3007, program mengelola posisi dan kapasitas antrian.

Konstruktor menginisialisasi array serta mengatur nilai awal, method isFull_3007() dan isEmpty_3007() memeriksa apakah Queue penuh/kosong berdasarkan size_3007, enqueue_3007() menambah elemen di posisi rear dengan operasi mod (rear + 1) % capacity agar indeks berputar secara melingkar, dequeue_3007() menghapus dan mengembalikan elemen dari front dengan menggeser pointer depan, metode front_3007() dan rear_3007() menampilkan nilai elemen terdepan dan terbelakang tanpa menghapus, sedangkan display_3007() mencetak isi Queue dari depan ke belakang.

2.2.3 QueueArrayDriver

```java
package pekan4_2511533007;

public class QueueArrayDriver_2511533007 {
        public static void main(String[] args) {
                QueueArray_2511533007 queue_3007 = new QueueArray_2511533007(1000);
                queue_3007.enqueue_3007(10);
                queue_3007.enqueue_3007(20);
                queue_3007.enqueue_3007(30);
                queue_3007.enqueue_3007(40);
                System.out.println("Item di depan " + queue_3007.front_3007());
                System.out.println("item paling belakang " + queue_3007.rear_3007());
                System.out.println("tampilan queue");
                queue_3007.display_3007();
                System.out.println();
                System.out.println(queue_3007.dequeue_3007() + " dihapus dari queue");
                System.out.println("item di depan:" + queue_3007.front_3007());
                System.out.println("item dibelakang: " + queue_3007.rear_3007());
                System.out.println("tampilan queue setelah satu data dihapus");
                queue_3007.display_3007();
        }

}
```

Kode Program 2.3

Kode QueueArrayDriver_2511533007 berfungsi sebagai *driver class* untuk menjalankan kelas QueueArray_2511533007 sebelumnya. Program dimulai dengan membuat objek Queue dengan kapasitas 1000, lalu menambahkan empat elemen (10, 20, 30, 40) menggunakan method enqueue_3007(). Selanjutnya, program menampilkan elemen terdepan dan terbelakang menggunakan front_3007() dan rear_3007(), mencetak isi antrian lewat display_3007(), kemudian menghapus satu elemen dari depan dengan dequeue_3007(), dan terakhir menampilkan kembali status Queue setelah penghapusan.

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan:20
item dibelakang: 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

gambar 2

2.2.4 QueueLinkedList

```java
package pekan4_2511533007;

import java.util.*;

public class QueueLinkedList_2511533007 {
	public static void main(String[] args) {
		Queue<Integer> q_3007 = new LinkedList<>();
		// tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
		for (int i = 0; i < 6; i++)
			q_3007.add(i);
		// Menampilkan isis antrian.
		System.out.println("Elemen Antrian " + q_3007);
		// Untuk menghaous kepala antrian.
		int hapus_3007 = q_3007.remove();
		System.out.println("Hapus elemen = " + hapus_3007);
		System.out.println(q_3007);
		// Untuk melihat antrian terdepan
		int depan_3007 = q_3007.peek();
		System.out.println("kepala Antrian = " + depan_3007);

		int banyak_3007 = q_3007.size();
		System.out.println("Size Antrian = " + banyak_3007);
	}

}
```

Kode Program 2.4

Program menginisialisasi Queue, lalu menambahkan enam elemen (0 sampai 5) secara berurutan menggunakan metode add() dalam perulangan, menampilkan seluruh isi queue, menghapus elemen terdepan dengan remove() yang mengembalikan dan menghapus nilai tersebut, menampilkan kembali queue setelah penghapusan, mengakses elemen terdepan tanpa menghapusnya melalui peek(), serta menampilkan jumlah elemen tersisa menggunakan size().

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
kepala Antrian = 1
Size Antrian = 5
```

gambar 3

2.2.5 ReverseData

```java
package pekan4_2511533007;

import java.util.*;

public class ReverseData_2511533007 {
	public static void main(String[] args) {
		Queue<Integer> q_3007 = new LinkedList<Integer>();
		q_3007.add(1);
		q_3007.add(2);
		q_3007.add(3); // [1, 2, 3]
		System.out.println("sebelum reverse" + q_3007);
		Stack<Integer> s_3007 = new Stack<Integer>();
		while (!q_3007.isEmpty()) { //Q->S
			s_3007.push(q_3007.remove());
		}
		while (!s_3007.isEmpty()) { // S->Q
			q_3007.add(s_3007.pop());
		}
		System.out.println("sesudah reverse = " + q_3007); // [3, 2, 1]
```

```
        }
}
```

Kode Program 2.5

Program membuat objek Queue<Integer> berbasis LinkedList dan menambahkan elemen 1, 2, 3 lalu print sebelum proses reverse. Selanjutnya, elemen antrian dipindahkan satu per satu ke dalam Stack<Integer> menggunakan loop while (!q_3007.isEmpty()) dengan remove() dari queue dan push() ke stack. Proses ini membalik urutan karena elemen terakhir yang masuk ke queue “3” akan berada di puncak stack. Setelah itu, elemen dikembalikan dari stack ke queue menggunakan pop() dari stack dan add() ke queue, sehingga urutan akhir menjadi [3, 2, 1].

```
sebelum reverse[1, 2, 3]
sesudah reverse = [3, 2, 1]
```

gambar 4

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Praktikum ini membahas implementasi struktur data Queue dalam Java melalui beberapa pendekatan dan studi kasus. Stack adalah struktur data yang bekerja dengan prinsip FIFO (*First In, First Out*), di mana elemen yang terakhir dimasukkan adalah yang pertama dikeluarkan.

.Implementasi queue dapat dilakukan melalui circular array dan LinkedList. Operasi dasar seperti enqueue (add()), dequeue (remove()), akses elemen terdepan (peek()/front()), dan pengecekan (isEmpty(), isFull()) telah berhasil dicoba. Selain itu, penggunaan queue dan stack secara bersamaan berhasil memanipulasi data agar dapat membalikkan elemen.

DAFTAR PUSTAKA

[1] GeeksforGeeks, "Introduction to Circular Queue," [Daring]. Tersedia pada: https://www.geeksforgeeks.org/dsa/introduction-to-circular-queue-1/. [Diakses: 30-Apr-2026].

[2] W3Schools, "Java Iterator," [Daring]. Tersedia pada: https://www.w3schools.com/java/java_iterator.asp. [Diakses: 30-Apr-2026].

[3] GeeksforGeeks, "Basic Operations for Queue Data Structure," [Daring]. Tersedia pada: https://www.geeksforgeeks.org/dsa/basic-operations-for-queue-in-data-structure/. [Diakses: 30-Apr-2026].

[4] GeeksforGeeks, "Queue Data Structure," [Daring]. Tersedia pada: https://www.geeksforgeeks.org/dsa/queue-data-structure/. [Diakses: 30-Apr-2026].

Laporan Tugas

a. Kode program ADT

```java
package pekan4_2511533007;

import java.util.*;

public class Queue_2511533007 {

    int front_3007, rear_3007, size_3007;
    int max_3007;
    String[] queue_3007;

    public Queue_2511533007(int capacity_3007) {
        this.max_3007 = capacity_3007;
        front_3007 = this.size_3007 = 0;
        rear_3007 = capacity_3007 - 1;
        queue_3007 = new String[this.max_3007];
    }

    public boolean isFull_3007() {
        return (this.size_3007 == this.max_3007);
    }

    public boolean isEmpty_3007() {
        return (this.size_3007 == 0);
    }

    public void enqueue_3007(String namaPelanggan_3007) {
        if (isFull_3007()) {
            System.out.println("Antrian penuh");
            return;
        }
        this.rear_3007 = (this.rear_3007 + 1) % this.max_3007;
        this.queue_3007[this.rear_3007] = namaPelanggan_3007;
        this.size_3007 = this.size_3007 + 1;
        System.out.println(namaPelanggan_3007 + " berhasil masuk antrian");
    }

    public String dequeue_3007() {
        if (isEmpty_3007())
            return null;
        String pelanggan_3007 = this.queue_3007[this.front_3007];
        this.queue_3007[this.front_3007] = null;
        this.size_3007 = this.size_3007 - 1;;
        return pelanggan_3007;
    }

    public void display_3007() {
        if (isEmpty_3007()) {
            System.out.println("\nAntrian kosong\n");
            return;
        }
        System.out.println("Antrian loket:");
        for (int i = 0; i < size_3007; i++) {
            int index_3007 = (front_3007 + i) % max_3007;
            System.out.print(queue_3007[index_3007]);
            if (i < size_3007 - 1) {
                System.out.print("\n");
            }
        }System.out.println();
    }

    public void reverse_3007() {
        if (isEmpty_3007()) {
            System.out.println("Antrian kosong");
            return;
        }

        Stack<String> stack_3007 = new Stack<String>();

        int jumlah_3007 = this.size_3007;
```

```java
        for (int i = 0; i < jumlah_3007; i++) {
            stack_3007.push(this.queue_3007[(this.front_3007 + i) % this.max_3007]);
        }

        this.front_3007 = 0;
        this.rear_3007 = max_3007 - 1;
        this.size_3007 = 0;

        while (!stack_3007.isEmpty()) {
            this.rear_3007 = (this.rear_3007 + 1) % this.max_3007;
            this.queue_3007[this.rear_3007] = stack_3007.pop();
            this.size_3007++;
        }
    }

    public String front_3007() {
        if (isEmpty_3007())
        return null;
        return this.queue_3007[this.front_3007];
    }

    public String rear_3007() {
        if (isEmpty_3007()) return null;
        return this.queue_3007[this.rear_3007];
    }
}
```

b. Kode Program

```java
package pekan4_2511533007;

import java.util.*;

public class AntrianLoket_2511533007 {

    static void tampilkanMenu_3007() {
        System.out.println("=== PROGRAM ANTRIAN LOKET ===");
        System.out.println("");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse Antrian");
        System.out.println("5. Keluar");
        System.out.println("");
        System.out.print("Pilih menu: ");
    }

    public static void main(String[] args) {
        Scanner sc_3007 = new Scanner(System.in);
        Queue_2511533007 antrian_3007 = new Queue_2511533007(10);
        int choice_3007;

        do {
            tampilkanMenu_3007();
            choice_3007 = sc_3007.nextInt();
            sc_3007.nextLine();

            switch (choice_3007) {
                case 1: //tambah antrian
                    if (antrian_3007.isFull_3007()) {
                        System.out.println("Antrian penuh, tidak bisa menambah pelanggan");
                    } else {
                        System.out.print("Masukkan nama pelanggan: ");
                        String nama_3007 = sc_3007.nextLine();
                        antrian_3007.enqueue_3007(nama_3007);
                    }
                    break;
                case 2: //hapus antrian
                    String hapus_3007 = antrian_3007.dequeue_3007();
                    if (hapus_3007 != null) {
                        System.out.println(hapus_3007 + "telah dihapus");
                    }
```

```
                    break;
                case 3: //tampilkan antrian
                    antrian_3007.display_3007();
                    break;

                case 4: //reverse antrian
                    System.out.println("Sebelum reverse: ");
                    antrian_3007.display_3007();
                    antrian_3007.reverse_3007();
                    System.out.println("Sesudah reverse: ");
                    antrian_3007.display_3007();
                    break;
                case 5: // keluar
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("pilihan tidak valid");
                    break;
            }

        } while (choice_3007 != 5);

        sc_3007.close();
    }
}
```

Penjelasan kode program

Program ini mengimplementasikan struktur data Queue berbasis array untuk mensimulasikan antrian loket. Kelas Queue_2511533007 menangani logika inti penyimpanan data string, yang mencakup operasi penambahan pelanggan (enqueue), penghapusan (dequeue), serta pembalik urutan queue (reverse) yang memanfaatkan bantuan struktur data Stack. Keseluruhan alur program dikendalikan oleh kelas AntrianLoket_2511533007 melalui interface menu yang dibuat dengan switch case, yang memungkinkan pengguna untuk memilih, memanipulasi, dan melihat kondisi queue hingga kapasitas maksimal sepuluh elemen.